

Banco de Dados II

Entrega: Polymarket no BigQuery

Análise de evento, inferência com SQL e alerta com EDA em .ipynb

Tema	Análise de mercados do Polymarket usando BigQuery
Arquivos principais	README.md, relatorio.ipynb, pasta de dados com arquivos em .csv (via google drive)
Entrega oficial	GitHub, na branch individual do trainee, com Pull Request para develop ao final

1. Objetivo

Nesta entrega, cada trainee deverá escolher um evento ou mercado específico do Polymarket e desenvolver uma análise no BigQuery. O foco é usar SQL para explorar dados, construir consultas analíticas, observar padrões, propor inferências e criar dois alertas em SQL (que não precisam focar exclusivamente em estratégias operacionais de compra/venda, podendo apenas sinalizar movimentações atípicas ou de volume no mercado, ou estar em conformidade com estratégias analíticas conhecidas como Rolling Windows, MACD, desvios estatísticos, etc.), com todo o estudo em um documento .ipynb (Jupyter Notebook).

O BigQuery será usado como ambiente de consulta e execução. O GitHub será usado como ambiente oficial da entrega, versionamento e revisão.

1.1. Acesso e Seleção do Projeto no BigQuery (Passo a Passo)

Para realizar as consultas analíticas, você utilizará o ambiente do Google Cloud Console e o BigQuery Studio. Cada trainee foi adicionado ao projeto com a permissão de Leitor de Dados (Data Viewer) e permissão de execução de consultas. Toda consulta deverá ser executada a partir do seu projeto pessoal (BigQuery Sandbox), referenciando o ID do projeto de treinamento no corpo do código SQL. Siga os passos abaixo para acessar e fixar o projeto:

1. Acessar o Console do Google Cloud:

- Faça login com seu email informado à Analytica e acesse o Google Cloud Console.

2. Selecionar o Projeto:

- No menu superior (barra de navegação cinza/azul), clique no dropdown de seleção de projetos (à esquerda do campo de busca, geralmente exibindo o nome de um projeto ativo ou "Selecione um projeto").
- Na janela que se abre, clique na aba Todos (ou *All*).
- No campo de busca, digite o ID do projeto do estudo: even-continuity-441808-j0.
- Clique no projeto correspondente na lista para selecioná-lo.

3. Entrar no BigQuery Studio:

- No canto superior esquerdo, clique no menu de navegação (três listras horizontais/hambúrguer).
- Role a página até a seção de Analytics e selecione BigQuery (ou simplesmente digite "BigQuery" na barra de pesquisa superior e selecione o serviço).

4. Fixar/Visualizar o Projeto no Explorer:

- a. Na barra lateral esquerda do BigQuery Studio, em Explorer, você deverá ver o ID `even-continuity-441808-j0` listado.
 - b. Caso ele não apareça automaticamente: Clique em + ADICIONAR (ou *+ ADD*) no topo do painel Explorer.
 - c. Selecione Marcar projeto com estrela por nome (ou *Star a project by name*).
 - d. Digite o ID do projeto: `even-continuity-441808-j0`.
 - e. Clique em Marcar com estrela (ou Star).
 - f. O projeto ficará fixado no seu Explorer. Ao expandi-lo, você terá acesso aos datasets `polymarket_optimized` e `polymarket_mart` para realizar suas queries.
5. Executar Consultas usando seu Limite Individual:
- a. Certifique-se de que o projeto selecionado no topo da tela do Console é o **seu projeto pessoal** (modo Sandbox, com 1 TB gratuito mensal).
 - b. Como você rodará as queries a partir do seu projeto, você deve obrigatoriamente referenciar o caminho absoluto das tabelas do treinamento no **FROM** da sua consulta, utilizando o ID do projeto entre crases:

```
SELECT

    market_id, question, event_title, volume, created_at, end_date, closed

FROM `even-continuity-441808-j0.polymarket_optimized.markets`

WHERE LOWER(question) LIKE '%liverpool%'

ORDER BY volume DESC

LIMIT 30;
```

6. Sobre suas Permissões:

- a. A role *Data Viewer* concede a você apenas acesso de leitura às tabelas e views do projeto `even-continuity-441808-j0`. Você **não** possui permissão de *Job User* nele, o que impede a execução direta de consultas dentro daquele ambiente. Ao rodar o SQL a partir do seu próprio projeto, puxando os dados de lá, você contorna o bloqueio e consome exclusivamente a sua cota individual. Salve os resultados localmente e armazene-os na pasta `dados/` de sua entrega.

Salve os resultados das suas explorações localmente (exportando como CSV na interface) na pasta `dados/`. **Atenção:** Devido aos limites de armazenamento, o repositório já está configurado via `.gitignore` para bloquear automaticamente o commit de arquivos `.csv`. Portanto, você não precisa se preocupar em isolá-los no Git, mas esses arquivos devem ser enviados para uma pasta no Google Drive. O notebook `.ipynb` deve continuar apontando para a pasta local `dados/` (ex: `pd.read_csv('dados/arquivo.csv')`), permitindo que o avaliador baixe os dados do seu Drive e os coloque na mesma pasta para reproduzir a análise.

2. Contexto da atividade

A atividade deve partir de uma pergunta clara sobre um evento real do Polymarket. A análise não deve ser apenas uma coleção de consultas soltas. Ela deve formar uma investigação sobre como o mercado se comportou antes, durante ou depois de um acontecimento relevante.

A entrega deve evitar afirmações fortes de causalidade sem evidência suficiente. O esperado é que o trainee proponha hipóteses, observe relações temporais e discuta limitações da análise.

3. Caso de exemplo

Exemplo de caso: mercado “Biden drops out in July?” no Polymarket.

Esse mercado tratava da possibilidade de Joe Biden anunciar sua retirada da disputa presidencial dos Estados Unidos até o fim de julho de 2024. Em 21 de julho de 2024, Biden anunciou que não seguiria como candidato à reeleição.

Uma análise possível seria investigar como o volume negociado, o preço/probabilidade implícita e a volatilidade do mercado se comportaram nos dias anteriores e posteriores ao anúncio.

Exemplos de perguntas para esse caso:

- O volume negociado aumentou antes do anúncio oficial?
- Houve mudança brusca na probabilidade implícita antes ou depois do evento externo?
- Quais foram os períodos de maior variação no preço do outcome?
- O mercado reagiu de forma gradual ou concentrada em poucos momentos?
- Que tipo de alerta poderia ser criado para identificar movimento anormal nesse mercado?

4. Estrutura esperada da entrega

A pasta individual deve seguir a estrutura abaixo:

```
entregaveis/
  banco_de_dados_2/
    nome_trainee/
      README.md
      relatorio.ipynb
      dados/ # # Pasta local para os CSVs (O repositório já está configurado para ignorar
arquivos .csv no git add)
```

5. Arquivos da entrega

5.1 README.md

O README deve apresentar a entrega e explicar como ela deve ser lida. Deve conter:

- nome do trainee;
- evento ou mercado escolhido;
- pergunta central da análise;
- hipótese inicial;
- tabelas ou datasets utilizados no BigQuery;
- período analisado;
- descrição breve dos arquivos SQL e do alerta final.
- link público do Google Drive contendo a pasta **dados/** com os CSVs utilizados na análise (garanta que o link esteja com permissão de leitura para "Qualquer pessoa com o link")

5.2 relatorio.ipynb

O notebook é o documento central da entrega: ele deve conter a análise completa, combinando texto explicativo, as queries utilizadas, os dados obtidos e as visualizações. Recomenda-se texto curto e objetivo, seguindo a progressão abaixo:

1. **Evento(s) escolhido(s), pergunta de pesquisa e hipótese inicial.**
2. **Exploração inicial:** as primeiras queries feitas no BigQuery, acompanhadas do CSV resultante de cada uma (o CSV completo, um recorte ou um HEAD, salvo em dados/). Essa etapa deve mostrar como o dataset foi explorado, o que foi encontrado de relevante e qual(is) mercado(s) foram selecionados para análise.
3. **Agregações e subqueries de interesse:** as consultas analíticas desenvolvidas a partir da exploração (apresentadas como queries normais, com CTEs/subqueries quando fizer sentido), junto com a lógica de otimização adotada para evitar custo computacional grande (ver seção 7).
4. **Dois Alertas Finais:** as queries consolidadas (ou query unificada), com 70 linhas ou mais no total, que implementam as regras dos dois alertas. Lembra-se que estes alertas não precisam ser necessariamente gatilhos operacionais de trading, podendo focar em sinalizar movimentações atípicas gerais de volume/preço ou conformar-se a alguma metodologia estatística/decisória conhecida (como Rolling Windows, MACD, modelos de desvio padrão ou score Z, etc.), retornando apenas os registros considerados relevantes.
5. **Demonstração:** uma visualização simples dos dados analisados, usando matplotlib ou outra biblioteca de interesse, onde seja possível observar os padrões que os alertas identificam, acompanhada de alguma métrica de assertividade para cada um deles (por exemplo: de todos os disparos do alerta no período, quantos correspondem a movimentos reais relevantes?).

6. Exemplo de lógica de alerta

Abaixo está uma estrutura conceitual. Os nomes de tabelas e colunas devem ser adaptados ao projeto real no BigQuery e ao model criado pelo trainee.

```
-- dbt/models/alerts/alerta_evento.sql

SELECT
    market_id,
    event_title,
    data_referencia,
    outcome,
    probabilidade_atual,
    probabilidade_anterior,
    variacao_probabilidade,
    volume_atual,
    volume_medio_recente,
    tipo_alerta
FROM {{ ref('mart_evento_polymarket') }}
WHERE
    tipo_alerta IS NOT NULL;
```

Exemplos de regras possíveis:

- variação brusca de probabilidade em curto intervalo;
- volume atual muito acima da média recente;
- queda relevante de liquidez;
- movimento forte logo após um evento externo;
- divergência entre volume alto e baixa mudança de preço.

7. Uso inteligente do BigQuery

A entrega deve demonstrar atenção ao custo e aos limites de consulta do BigQuery. O dataset possui tabelas de granularidade transacional muito grandes, por isso o uso correto de partições, clustering e indexações é essencial para evitar o estouro da cota mensal (1 TB gratuito).

7.1 Boas Práticas Gerais

- **Evitar `SELECT *`:** Sempre selecione apenas as colunas estritamente necessárias para a análise.
- **Uso de `LIMIT`:** Use a cláusula `LIMIT` apenas para inspeção estrutural inicial. O `LIMIT` não reduz o custo de processamento em tabelas colunares (BigQuery cobra pelas colunas inteiras varridas).
- **Datas Literais:** Use datas literais fixas (ex: `'2026-04-07'`) em vez de funções voláteis como `CURRENT_DATE()` ou `CURRENT_TIMESTAMP()`. Como o dataset é estático, o uso de constantes permite que o BigQuery aproveite o cache de resultados gratuito em execuções sucessivas.
- **Tabelas de Mart:** Sempre prefira as tabelas agregadas (`polymarket_mart.daily_market_stats` ou views como `v_market_daily`) para fazer estudos de baseline e tendências diárias. Elas processam megabytes em vez de gigabytes.

7.2 Otimizações de Performance e Custo (Granularidade Fina)

Particionamento e Clustering:

- A tabela detalhada de transações (`polymarket_optimized.trades`) é particionada por dia (`trade_timestamp`) e clusterizada por `market_id`. Para consultá-la com custo mínimo:
- Sempre forneça um filtro de data explícito no `WHERE` usando `DATE(trade_timestamp)`.
- Sempre filtre pelo `market_id` do mercado escolhido. O BigQuery usará os clusters para ignorar os dados de outros mercados na partição, reduzindo drasticamente o volume lido.

Podamento de Partição com `OR` vs. `BETWEEN`:

- Ao fazer estudos de múltiplos momentos ou datas específicas distantes (por exemplo, analisar a véspera de 3 jogos diferentes), evitar filtros amplos como `DATE(trade_timestamp) BETWEEN '2025-11-01' AND '2026-05-01'`. Isso força o BigQuery a ler todas as partições intermediárias.
- Em vez disso, utilize o podamento preciso de partições concatenando intervalos curtos com a cláusula `OR` (ex: `(DATE(trade_timestamp) BETWEEN '2026-01-24' AND '2026-01-25') OR (DATE(trade_timestamp) BETWEEN '2026-04-07' AND '2026-04-08')`). Isso faz o BigQuery ler estritamente as partições declaradas, economizando mais de 90% do custo de scan.

Ordem de Sintaxe SQL:

- No BigQuery (GoogleSQL), a cláusula `WINDOW` deve vir obrigatoriamente após a cláusula `QUALIFY` (caso use filtragem de janela inline).
- Para evitar erros de sintaxe complexos e melhorar a legibilidade, prefira declarar as funções de janela em uma Common Table Expression (CTE) e realizar a filtragem com um `WHERE` simples no bloco principal.

8. Regras gerais

- Cada trainee deve escolher um evento ou mercado específico do Polymarket (ou mais).
- Cada arquivo SQL deve conter comentários explicando a pergunta ou etapa representada pela consulta.
- As queries devem ser executáveis no BigQuery.
- A entrega deve conter uma análise, não apenas consultas isoladas.

- O relatório deve separar observação dos dados, hipótese e limitação.
- As queries dos dois alertas devem retornar apenas os registros considerados relevantes pelas regras definidas.
- Não versionar credenciais, tokens ou arquivos pessoais de ambiente com dados sensíveis.

9. Checklist antes de finalizar

- ☐ **README.md** apresenta evento, pergunta, hipótese, tabelas usadas e o resumo do resultado final (os dois alertas e seus escopos).
- ☐ **relatorio.ipynb** contém a análise completa: exploração inicial com CSVs, agregações, os dois alertas e visualização.
- ☐ Os arquivos **.csv** não subirão para o GitHub (bloqueados pelo **.gitignore**).
- ☐ O link do Google Drive com os CSVs foi adicionado ao **README.md**.
- ☐ O código do **relatorio.ipynb** aponta corretamente para a pasta local **dados/** para a leitura dos arquivos
- ☐ As queries dos dois alertas finais têm 70+ linhas no total e retornam apenas registros relevantes.
- ☐ A visualização permite observar os padrões identificados pelos alertas e há métricas de assertividade para cada um.
- ☐ As queries finais evitam varrer dados desnecessários.
- ☐ Nenhum arquivo com credenciais foi versionado.
- ☐ A pasta individual está organizada.
- ☐ O commit será feito na branch individual do trainee.

11. Git, branch e Pull Request

A entrega deve ser commitada na branch individual de cada trainee. Não faça commit direto na branch develop.

Antes de commitar, confira se você está na sua branch:

```
git status
```

Adicione exatamente a sua pasta individual:

```
git add entregaveis/banco_de_dados_2/nome_trainee/
```

Faça o commit da entrega final:

```
git commit -m "Finaliza entrega de Banco de Dados II"
```

Suba sua branch para o GitHub:

```
git push
```

Depois do push, abra um Pull Request da sua branch individual para a branch develop. O PR deve ser aberto apenas depois que a entrega estiver concluída.

12. Resultado esperado

A entrega final deve mostrar que o trainee consegue usar SQL no BigQuery para investigar um evento real do Polymarket, construir consultas analíticas, interpretar padrões e transformar a análise em regras de alerta.

O resultado deve ser uma pasta individual organizada, com um notebook de análise completo, consultas executáveis, evidências da exploração inicial (CSVs via Google Drive) , queries de dois alertas claros e conectados à análise feita, e visualizações que demonstrem os padrões identificados.

13. Fontes usadas no exemplo e na orientação

- Polymarket: mercado “Biden drops out in July?”.
- Anúncio público de Joe Biden em 21 de julho de 2024 sobre sua retirada da disputa presidencial.